```
  result

header = (response, status, contentType='text/plain;charset=utf-8')
->
  response.writeHead status, 'Content-Type': contentType

httpRequestMatch = (request, method) -> request.method is method
isGet = (request) -> httpRequestMatch request, "GET"
isPost = (request) -> httpRequestMatch request, "POST"

render = (response, content) ->
  header response, 200
  response.end content, 'utf8'

renderAsJson = (response, object) -> render response,
JSON.stringify object

notFound = (response) ->
  header response, 404
  response.end 'not found', 'utf8'

handleProfitRequest = (request, response, price, costPrice,
overhead) ->
  valid = (price) -> price and /^[0-9]+$/gi.test price
  if valid price
    renderAsJson response, profit: profit price, costPrice,
overhead
  else
    renderAsJson response, profit: 0

makeDbOperator = (db) ->
  (operation) ->
    (entry, params...) ->
      db[operation] entry, params...

makeRequestHandler = (load, save) ->
  rendersIfFound = (response) ->
    (error, data) ->
      if error
        notFound response
      else
        renderAsJson response, data

  (request, response, name) ->
    if isGet request
      load name, rendersIfFound response
    else if isPost request
      withCompleteBody request, ->
        save name, rendersIfFound response
    else
```